

SILAB[®]

Version 7.1
Handbuch Szenariengestaltung

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1 EINLEITUNG	4
2 UMWELTBEDINGUNGEN	6
2.1 Tageszeiten, Schatten	6
2.2 Nachtfahrten	6
2.3 Nebel	7
2.4 Regen / Gewitter	9
2.5 Winter / Schnee	12
3 LANDSCHAFTS- /STRASSEN GESTALTUNG.....	15
3.1 Wald	15
3.2 Tunnel	16
3.3 Brücke	21
4 UMGEBENDER VERKEHR	24
4.1 Gegenverkehr auf der Landstraße	24
4.1.1 TrafficFlow (Quelle nicht ortsfest)	24
4.1.2 Einzelfahrzeuge	25
4.2 Vorfahrendes Fahrzeug	25
4.2.1 Vorfahrendes abbiegendes Fahrzeug auf der Landstraße	25
4.2.2 Wanderbaustelle auf der Autobahn	26
4.3 Stau auf zweispuriger Autobahn	26
4.4 Stehende Fahrzeuge	28
4.4.1 Pannenfahrzeug auf der Autobahn	28
4.4.2 Parkendes Fahrzeug	28

1 EINLEITUNG

Dieses Dokument wendet sich an SILAB-Anwender, die bereits mit Grundlagen der Konfiguration und Bedienung von SILAB und dem Entwurf von Strecken für SILAB vertraut sind. Je nach Themengebiet werden Kenntnisse, die in den Dokumenten

- Handbuch SILAB Essential
- Handbuch SILAB Professional
- Referenz SILAB-Konfiguration und Bedienung
- Referenz Datenbasis SCNX
- Referenz Verkehrssimulation TRFX
- Referenz Fußgängersimulation MOV

vermittelt werden, vorausgesetzt.

Anhand von kleinen Ausschnitten von Konfigurationsdateien wird in diesem Handbuch verdeutlicht, wie abwechslungsreiche Szenarien gestaltet werden können und wie bestimmte Situationen umgesetzt werden können.

Bei Bedarf wird auf die entsprechenden Referenzhandbücher verwiesen, wo zusätzliche Details nachgelesen werden können.

Ausgehend von einer (unvollständigen!) Basis-Konfigurationsdatei werden an folgende Stellen Konfigurationsabschnitte eingesetzt:

```
SILAB Configuration
{
    Computerconfiguration Rechner
    {
        include System\CompBase.inc

        # <<Computer-Konfiguration>>
    };
    DPUConfiguration DPUs
    {
        include System\DPUBase_SGE.inc

        Pool Aufzeichnung : Full
        {
            Executable = true;

            # <<DPU-Konfiguration>>
        };
    };
};

SILAB System
{
    # <<Systemabschnitt>>
};
include system\SYSBase.inc

include MOV/MOP_600.inc
include SGE/SGE_600.cfg
include scn\SCNXSGE_600.cfg
include snd\SNDX.cfg

SILAB SCNXSGE
{
    # <<SCNXSGE-Abschnitt>>
};
```

```
SILAB TRF
{
    # <<TRF-Definition>>
};

SILAB SCN
{
    include scnx\SCNX_600.cfg

    # <<SCN-Abschnitt>>

    define Module MeinModul
    {
        ModuleID = 3;

        # <<Modul>>

        define Course U1
        {
            NodeID = 1;
            Type = Landstrasse;

            # <<Course>>
        };
    };

    Map Map1
    {
        # <<Karte>>
    };
};
```

2 UMWELTBEDINGUNGEN

2.1 Tageszeiten, Schatten

Unterschiedliche Tageszeiten können über die Environment-Einstellungen in der DPU-Konfiguration eingestellt werden. In der Morgen- bzw. Abenddämmerung wird die Farbe des Sonnenlichts automatisch auf gelblich bzw. rötlich umgestellt.

- In der **DPU-Konfiguration** kann Jahreszeit (Monate von 0 bis 12) und Tageszeit (Stunden von 0 bis 24) gesetzt werden:

```
Environment.InTimeOfYear = 4;
Environment.InTimeOfDay = 17;
```
- Die Tageszeit kann auch während der Fahrt über Hedgehogs im **Course**-Abschnitt geändert werden:

```
Hedgehogs =
{
    # (Streckenmeter, Richtung, SpurID, Family, Name, Wert
      (1000, DontCare, 0, Env, CRTIMEOfDay, 0.1, TimeOfDay, 0.0)
};
```

Dabei gibt der Wert nach *TimeOfDay* die gewünschte neue Tageszeit an, der Wert nach *CRTIMEOfDay* gibt die Änderungsrate mit der diese eingestellt wird [1/s].
(→ Dokumentation *DPUEnvSCNX*)
- Durch die Einstellung von *SGEWorld.RenderMode* in der **DPU-Konfiguration** können Schatten aktiviert oder deaktiviert werden (dies kann nicht während der Fahrt umgeschaltet werden):

```
SGEWorld.RenderMode = 4; # aktiviert Schatten
SGEWorld.RenderMode = 2; # keine Schatten
```

(→ Dokumentation *DPUSGEWorld*)

2.2 Nachtfahrten

SILAB unterstützt zwei unterschiedliche Modi zur Darstellung von Nachtfahrten.

Diese werden über den Parameter *SGEWorld.RenderMode* in der **DPU-Konfiguration** ausgewählt.

- Für Wechsel zwischen Tag- und Nachtfahrt in derselben Strecke besonders geeignet ist der Modus „Daytime with Headlight“:

```
SGEWorld.RenderMode = 4; # mit Schatten tagsüber
SGEWorld.RenderMode = 2; # keine Schatten
```

(→ Dokumentation *DPUSGEWorld*)
 - Hierbei wird eine geglättete, schönere Darstellung der Objekte insbesondere tagsüber erreicht.
 - Allerdings wird bei der Nachtfahrt nur das Licht der Scheinwerfer des Ego-Fahrzeugs dargestellt, nicht hingegen das von Straßenlampen oder Fremdfahrzeugen.
 - „Daytime with Headlight“ eignet sich also besonders für Strecken, in denen Tag- und Nachtfahrt kombiniert werden sollen und die Nachtfahrt außerorts durchgeführt wird.
- Eher für reine Nachtfahrten geeignet ist der Modus „Nighttime“:

```
SGEWorld.RenderMode = 5; # keine Schatten
```

```
SGEWorld.RenderMode = 6; # mit Schatten (nur tagsüber)
(→ Dokumentation DPUSGEWorld)
```

- Die Darstellung von Objekten wie z.B. Bäumen und Verkehrsschildern ist hier kantiger.
- Für die Nachtfahrt wird das Licht der Scheinwerfer von Ego- und Fremdfahrzeugen sowie von Straßenlampen berücksichtigt.
- Der Darstellungsmodus „Nighttime“ ist extrem rechenaufwändig! Es muss daher anhand der Anzeige „Ansicht → Simulations-Status“ in SILAB geprüft werden, dass die Grafik (SGE) nicht überlastet ist. Vermutlich ist eine Reduktion der Kantenglättung nötig:

```
SGEView.AntiAlias = 2; # in der DPU-Konfiguration
```

Zusätzlich zum Darstellungsmodus muss die Tageszeit eingestellt werden, siehe hierzu Kap. 2.1.

Einen visuellen Vergleich der beiden Darstellungsmodi ermöglicht die folgende Abbildung.



2.3 Nebel

Ein ringsum einheitlicher Nebel wird durch die Einstellung des Umweltparameters Humidity erreicht.

- In der **DPU-Konfiguration** wird hierzu eingefügt:

```
Environment.InHumidity = 0.8;
```

 Gültige Werte reichen von 0.1 für einen klaren Tag bis zu 1.0 für Nebel mit 50 m Sichtweite.

- Die Nebeldichte kann auch während der Fahrt über Hedgehogs im **Course**-Abschnitt geändert werden:

```
Hedgehogs =
{
    # (Streckenmeter, Richtung, SpurID, Family, Name, Wert
      (1000, DontCare, 0, Env, CRHumidity, 0.1, Humidity, 0.0)
};
```

Dabei gibt der Wert nach *Humidity* die gewünschte neue Luftfeuchtigkeit an, der Wert nach *CRHumidity* gibt die Änderungsrate mit der diese eingestellt wird [1/s].
(→ Dokumentation *DPUEnvSCNX*)



Abbildung 1 Ringsum einheitlicher Nebel

Unabhängig davon können Bodennebel-Felder platziert werden.

Die Bodennebel-Objekte werden mit einem Landschaftsgenerator im **Course**-Abschnitt definiert:

```
LandscapeTypeExtra GroundFog
{
    Left = false;
    MinObjdist = -30;
    MaxObjdist = 30;

    MinObjpos = 150;
    MaxObjpos = 250;
    Objangle = 0.2;
};
```

Diese Bodennebel-Felder müssen sparsam verwendet werden, da sie sehr rechenaufwändig sind.



Abbildung 2 Bodennebel

2.4 Regen / Gewitter

- In der **DPU-Konfiguration** kann Niederschlag ausgelöst werden:
`Environment.InPrecipitation = 0.7; # Wertebereich 0 bis 1`
 Falls die Soundsimulation SNDX eingesetzt wird, führt dies automatisch dazu, dass ein Regengeräusch aktiviert wird.
- Unwetter können durch die zusätzliche Aktivierung von Blitz und Donner dargestellt werden:
`Environment.InThunderstorm = 0.8; # Wertebereich 0 bis 1`
- Für einen realistischeren Eindruck sollte der Himmel bedeckt sein. Dazu kann ein anderes Panorama gewählt werden. Im **SCNXSGE-Abschnitt** muss dazu folgendes eingetragen werden:


```
SCNXSGE Panorama
{
    # Panoramatextur
    Texture = environment.surrounding.hills1rainy;
};
```
- Alternativ kann auch das Panorama `environment.surrounding.hills1overcast` verwendet werden.
- Das Regenwetter-Panorama erfordert auch eine geänderte Beleuchtung der Szene. Dies kann durch die Anweisung
`Environment.InLightingMode = 2; # Wertebereich 0 bis 2`
 in der DPU-Konfiguration ausgelöst werden.
- Allerdings können die Panoramen und auch der `LightingMode` nicht während der Simulation ausgetauscht werden. Soll also ein kurzer Regenabschnitt in eine längere Fahrt bei schönem Wetter eingefügt werden, muss der bedeckte Himmel stattdessen

durch Nebel simuliert werden:

```
Environment.InHumidity = 0.5;
```

- Für die Fahrzeuge des Fremdverkehrs TRFX kann auch dargestellt werden, dass Wassertropfen von der nassen Straße aufgewirbelt werden. Dazu muss der DPU-Parameter `TRF.EnableSpray = true;` gesetzt werden. Die Stärke der Gischt wird automatisch anhand von Regendichte und Fahrtgeschwindigkeit festgelegt.
- Die Environment-Variablen können auch durch Hedgehogs im Course-Abschnitt während der Fahrt angepasst werden, vgl. dazu Kap. 2.1 und die → Dokumentation DPUEnvSCNX.
- Bei starkem Regen sollte der Eindruck erweckt werden, dass die Straße nass ist. Dazu kann das Objektband *WetStreet* verwendet werden.

```
Strip WetStreet
{
    YOffset = 0;
    Width = 8;

    Entry = 0;
    Start= 100;
    End = 690;
    Exit = 790;
};
```

- Das Objektband muss im **Course**-Abschnitt eingetragen werden. Der Parameter *Width* sollte etwas größer als die Gesamtbreite der Straße sein.
- Die Möglichkeit, zusätzliche Objektbänder im Course-Abschnitt einzutragen, wurde in SILAB 3.0 eingeführt und ist in der → Referenz Datenbasis SCNX beschrieben.



Abbildung 3 Regenwetter

2.5 Winter / Schnee



Abbildung 4 Winterlandschaft

Um die Darstellung der Umgebung von Sommer auf Winter umzustellen, müssen folgende Änderungen vorgenommen werden:

- In der **DPU-Konfiguration** muss die Jahreszeit auf „Winter“ umgestellt werden, z.B.
`Environment.InTimeOfYear = 0;`
- Die Beleuchtung muss an die durch den Schnee hellere Umgebung angepasst werden:
`Environment.InLightingMode = 1;`
 (→ Dokumentation *DPUEnvSCNX*)
- Im **Systemabschnitt** muss der „Winter“-Pfad gesetzt werden;
`PathODBAternate = winter;`
 (→ Dokumentation *Referenz Visualisierung SGE*)
- Der für die Straße verwendete `CourseType` kann so angepasst werden, so dass das Bankett entfernt wird und stattdessen Schneehaufen entlang dem Straßenrand platziert werden:

Dazu wird die Superstructure `SnowBorder` im **SCN-Abschnitt** definiert:

```
SCN Params
{
  SCN CrossSection
  {
    SCN Superstructures
    {
      # Schneestreifen
```

```

Strip SnowBorder {
    Smooth = true;
    Element Side1 {
        TextureName = "ROAD.SUPERSTRUCTURES.snow_border";
        Texturey1 = 0.7;
        Texturey2 = 1;
        Vy1 = -0.15;
        Vz1 = 0.08;
        Vy2 = -0.3;
        Vz2 = 0;
        TextureWidth = 8;
    };
    Element Top {
        TextureName = "ROAD.SUPERSTRUCTURES.snow_border";
        Texturey1 = 0.3;
        Texturey2 = 0.7;
        Vy1 = 0.15;
        Vz1 = 0.08;
        Vy2 = -0.15;
        Vz2 = 0.08;
        TextureWidth = 8;
    };
    Element Side2 {
        TextureName = "ROAD.SUPERSTRUCTURES.snow_border";
        Texturey1 = 0;
        Texturey2 = 0.3;
        Vy1 = 0.3;
        Vz1 = 0;
        Vy2 = 0.15;
        Vz2 = 0.08;
        TextureWidth = 8;
    };
};
};
};
};
};

```

Diese wird dann im CourseType anstelle der StoneBorder eingebunden:

```

CourseType Landstrasse
{
    # distgerade, distanz, höhe, textur, transparenz
    Shoulder = {1.0, 1.4, -0.7, Shoulder1, 1};

    ve = 70;
    LaneInfos =
    {
        (0, 3.5, 0, Towards, Tar, 1.0, 0.0, {(None, 0.0)}),
        (1, 3.5, 0, Onwards, Tar, 1.0, 0.0, {(None, 0.0)})
    };
    LaneTransitions =
    {
        (0, 0,
        {
            (StoneBorderLeft, -0.68, 1.5, Towards, 8.0, 0),
            (SnowBorder, 0.1, 1, Onwards, 8.0, 0),
            (ReflectionPosts, -0.2, 1.0, Towards, 50.0, 0.0),
            (Line, 0.12, 0.15, Towards, 1.0, 0.0)
        }
        ),
        (0, 1, {(Line, 0.0, 0.15, Towards, 2.0, 6.0)}),
        (1, 1,
        {
            (StoneBorderRight, 0.75, 1.5, Towards, 8.0, 0),
            (SnowBorder, -0.1, 1, Onwards, 8.0, 0),
            (ReflectionPosts, 0.2, 1.0, Onwards, 50.0, 0.0),

```

```
        (Line, -0.12, 0.15, Towards, 1.0, 0.0)
    }
)
};
};
```

Ein Beispiel für eine Winterlandschaft ist die *SILABDemo_Country_Winter.cfg* in der SILAB-Demo.

3 LANDSCHAFTS- /STRASSENGESTALTUNG

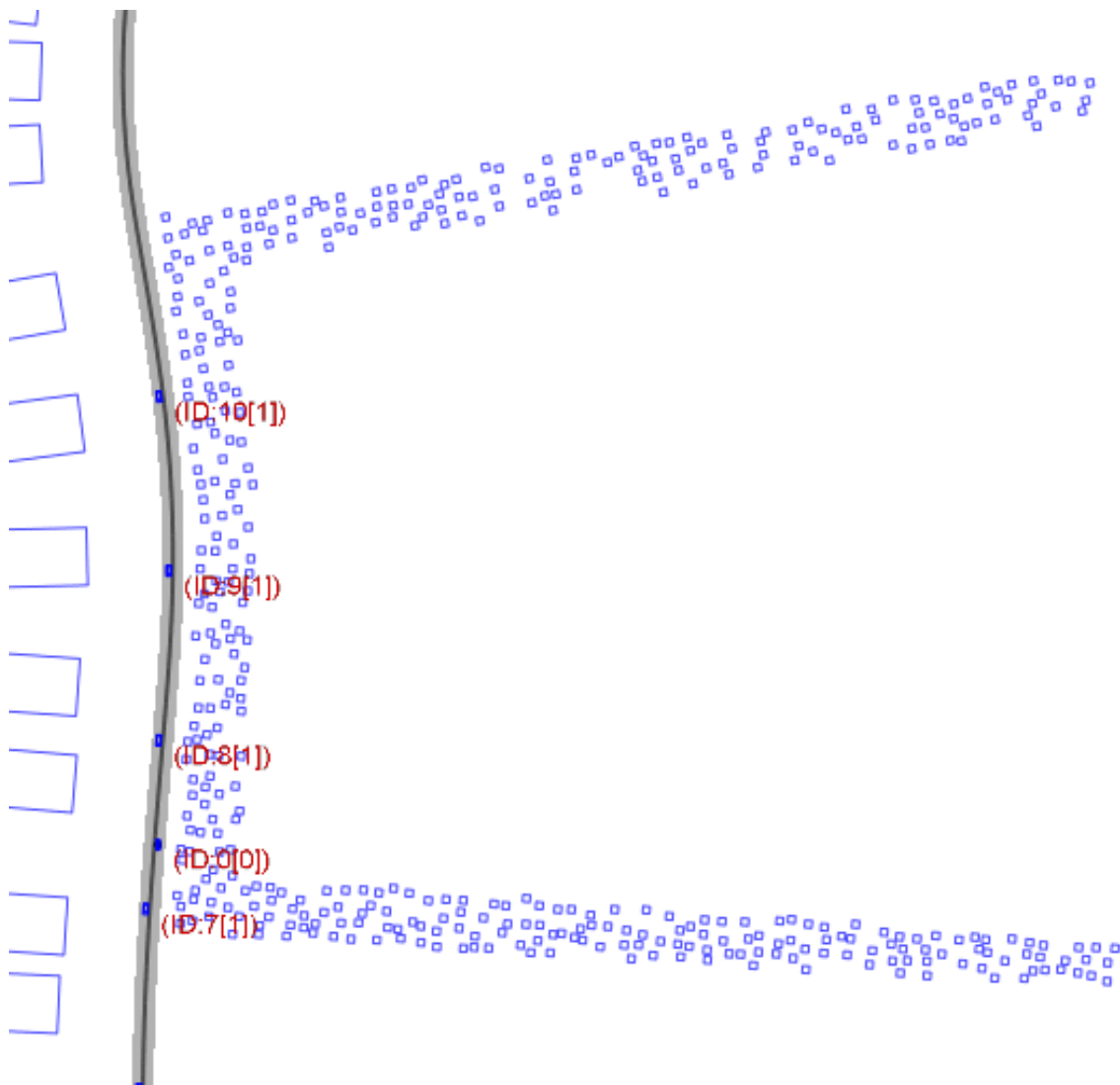
3.1 Wald

Wälder sind sehr gut geeignet, um Abwechslung in die Landschaft zu bringen, zudem wirken sie als Sichtschutz.

Waldbäume werden im **Course**-Abschnitt mit dem Landschaftsgenerator Forest (für Nadelbäume) bzw. LeafForest (für Laubbäume) platziert.

Da eine hohe Anzahl von Bäumen eine hohe Rechenlast erzeugt, ist es wichtig die Bäume möglichst effizient einzusetzen.

Der Landschaftsgenerator erzeugt daher anstelle eines großen, geschlossenen Waldgebiets („Rechteck“) automatisch einen Wald in „U-Form“:



Aus Sicht des Fahrers ist der Trick nahezu nicht sichtbar:



Der zugehörige Konfigurationsabschnitt lautet:

```
# Wald
LandscapeTypeExtra Forest
{
    Left = false;

    MinObjdist = 0;
    MaxObjdist = 200;

    MinObjpos = 200;
    MaxObjpos = 500;
};
```

3.2 Tunnel

Derzeit werden in SILAB Straßentunnel nur außerorts (auf Course-Streckentypen) unterstützt.

Die Definition eines Tunnels wird in drei Schritten durchgeführt:

1. Ein spezieller *CourseType* wird definiert, der Tunnelobjekte enthält
2. Der Landschaftsgenerator *Tunnel* wird eingefügt, der die Tunnelwände platziert
3. Die umgebende Landschaft wird angehoben
4. Grafische Objekte für die Tunnelportale werden platziert.

CourseType-Definition

Der Tunnelabschnitt erhält einen speziellen *CourseType*, der um „tunneltypische“ Elemente ergänzt wird: *TunnelVentilation* stellt sich drehende Ventilatoren dar. Im SCN-Abschnitt wird also folgende Definition eingefügt:

```
CourseType Landstrasse_Tunnel
{
    # distgerade, distanz, höhe, textur
    Shoulder = {1.0, 1.0, -0.7, Concrete1};

    GraphicOptions = {NormallyDark, ObjectsCourseRelative};
```



```

ve = 70;
LaneInfos =
{
  (0, 3.5, 0, Towards, Tar, 1.0, 0.0, {(None, 0.0)}),
  (1, 3.5, 0, Onwards, Tar, 1.0, 0.0, {(None, 0.0)})
};
LaneTransitions =
{
  (0, 0, {
    (Line, 0.12, 0.15, Towards, 1.0, 0.0)
  }),
  (0, 1, {
    (Line, 0.0, 0.15, Towards, 2.0, 0.0),
    (TunnelVentilation, 0, 1, Onwards, 60, 18)
  }),
  (1, 1, {
    (Line, -0.12, 0.15, Towards, 1.0, 0.0)
  })
};
};

```

Course-Definition

Der Streckenverlauf des Tunnels kann beliebig aus Kurven- und Geradenstücken zusammengesetzt werden. Das Landschafts-Höhenprofil wird so eingestellt, dass auf beiden Seiten der Straße die Landschaft (mindestens) 10 m höher als die Straße liegt (Offset = 10).

Fügen Sie also im **Course-Abschnitt** folgendes ein:

```

LandscapeTypeLeft <<beliebig>>
{
  Offset = 10;
  <<weitere Parameter wie gewünscht>>
};

LandscapeTypeRight <<beliebig>>
{
  Offset = 10;
  <<weitere Parameter wie gewünscht>>
};

```

Grafische Objekte

Im Verlauf des Tunnels muss noch eine Landschaftsoberfläche oberhalb des Tunnels dargestellt werden. Hierzu wird ein zusätzlicher Objektstreifen innerhalb des **Course-Abschnittes** definiert. Alle Beispiele beziehen sich auf einen Course mit Streckenlänge von 400 Metern.

```

Strip TunnelTop_8
{
  Orientation = Onwards;
  YOffset = 0;

  Start = 5;
  End = 395;
};

```

Die Tunnelwände werden mit Hilfe eines speziellen Landschaftsgenerators platziert.

```

LandscapeTypeExtra Tunnel
{
  Left = false;
  MinObjdist = -7.0;
  MaxObjdist = 0.0;
  MinObjpos = 6;
  MaxObjpos = 390;
};

```

```
};
```

Am Anfang und Ende des Tunnels fehlt nun noch das Tunnelportal. Zusätzlich wird auf den ersten Metern des Tunnels die Beleuchtung verstärkt, um dem Fahrer die Adaption zur hellen Umgebung zu erleichtern. Der Objects-Abschnitt muss ebenfalls innerhalb des **Course-Abschnittes** eingetragen werden.

```
Objects = {
  # Eingangsportal
  (environment.road.tunnel_entrance, 0, 0),
  (environment.road.tunnel_entry_hill_left, 0, 0),      #optional
  (environment.road.tunnel_entry_hill_right, 0, 0),     #optional

  # Beleuchtung
  (SCNX.ROAD.SUPERSTRUCTURES.OBJECTS.tunnel_lamp, 0, 0), # am Anfang
  (SCNX.ROAD.SUPERSTRUCTURES.OBJECTS.tunnel_lamp, 396, 0), # am Ende

  # Endportal
  (environment.road.tunnel_entrance, 396, 0, 180),
  (environment.road.tunnel_entry_hill_left, 396, 0, 180),      #optional
  (environment.road.tunnel_entry_hill_right, 396, 0, 180)     # "
};
```

Autobahntunnel

Für eine Autobahn mit zwei Fahrspuren pro Fahrtrichtung können ähnliche Definitionen verwendet werden. Solche Tunnel erhalten stets eine separate Röhre pro Fahrtrichtung, die jeweils eine ähnliche Größe hat wie die des Landstraßentunnels. Die CourseType-Definition kann dann wie folgt aussehen:

```
CourseType Autobahn_Tunnel
{
  ve = 150.0;

  Shoulder = {1.0, 1.0, -0.7, Concretel};

  LaneInfos =
  {
    (0, 0.5, 0.0, Towards, Concretel, 1.0, 0.0, { (None, 0.0) } ),
    (1, 3.75, 0.0, Towards, Tar, 1.0, 0.0, { (None, 0.0) } ),
    (2, 3.75, 0.0, Towards, Tar, 1.0, 0.0, { (None, 0.0) } ),
    (3, 3.0, 0.0, Towards, Concretel, 1.0, 0.0, { (None, 0.0) } ),
    (4, 3.75, 0.0, Onwards, Tar, 1.0, 0.0, { (None, 0.0) } ),
    (5, 3.75, 0.0, Onwards, Tar, 1.0, 0.0, { (None, 0.0) } ),
    (6, 0.5, 0.0, Onwards, Concretel, 1.0, 0.0, { (None, 0.0) } )
  };

  LaneTransitions =
  {
    (0, 1, {(Line, 0.0, 0.3, Towards, 1.0, 0.0)}),
    (1, 2, {(Line, 0.0, 0.15, Towards, 6.0, 12.0),
            (TunnelInside_2, 0, 1, Towards, 0, 0),
            (TunnelLight, 0, 1, Onwards, 30, 0),
            (TunnelVentilation, 0, 1, Onwards, 48, 0)
           }),
    (2, 3,
     {
       (Line, -0.3, 0.3, Towards, 1.0, 0.0),
       (TunnelTop_8_Wide, 1.5, 1, Towards, 0, 0)
     }
    ),
    (3, 4,
```

```

        {
            (Line, 0.3, 0.3, Onwards, 1.0, 0.0)
        }
    ),
    (4, 5, {(Line, 0.0, 0.15, Onwards, 6.0, 12.0),
            (TunnelInside_2, 0, 1, Towards, 0, 0),
            (TunnelLight, 0, 1, Onwards, 30, 0),
            (TunnelVentilation, 0, 1, Onwards, 48, 0)
        }
    ),
    (5, 6, {(Line, 0.0, 0.3, Onwards, 1.0, 0.0)})
};

```

Für die Tunnelportale werden die grafischen Objekte `tunnel_entry_motorway_{left,right}` verwendet, hier beispielhaft für einen 700 m langen Tunnel:

```

Objects =
{
    (environment.road.tunnel_entry_motorway_left, 0, -5.25),
    (environment.road.tunnel_entry_motorway_right, 0, 5.25),
    (environment.road.tunnel_entry_motorway_left, 699.9, 5.25, 180),
    (environment.road.tunnel_entry_motorway_right, 699.9, -5.25, 180)
};

```

DPU-Konfiguration

Damit die Beleuchtung des Tunnels korrekt simuliert wird, muss in der DPU-Konfiguration `SGEWorld.RenderMode = 5` (für Nachtfahrten) oder `SGEWorld.RenderMode = 6` (für Tagfahrten) gesetzt werden. Bitte beachten Sie allerdings, dass diese Darstellungsmodi sehr rechenaufwändig sind (→ Dokumentation *DPUSGEWorld*)



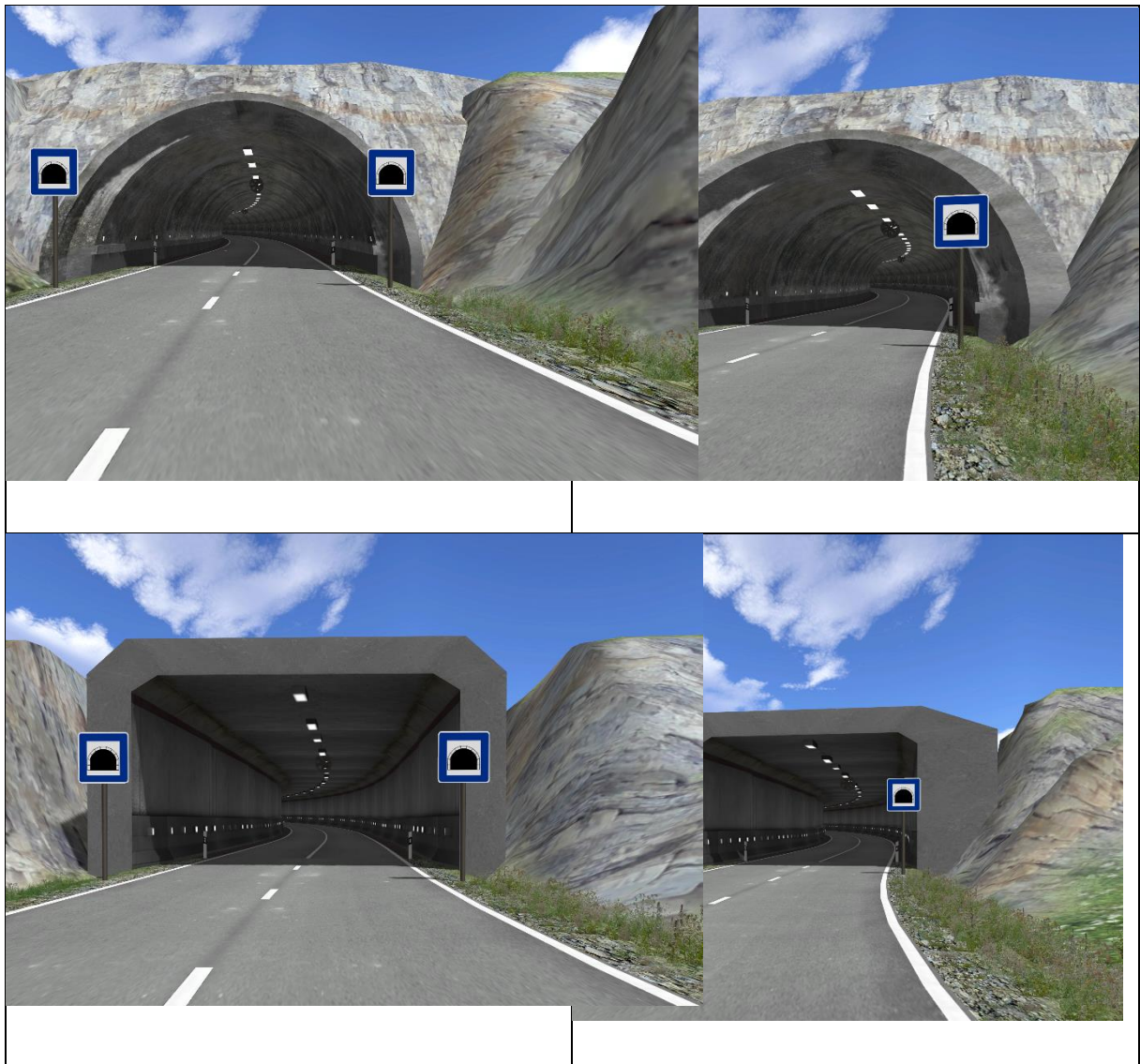
Abbildung 5 Tunneleinfahrt bei Nacht

Tunnelarten

In SILAB sind vier Arten von Tunneln integriert.

Tunnel	Breite innen/außen	Geeignet für:	Landschaftstyp
rund	9 / 11 m	2-spurige Straßen	Tunnel
rund breit	9 / 13.6 m	2-spurige Straßen mit Parkbuchten	TunnelWide
rechteckig	9 / 11.6 m	2-spurige Straßen in eine Richtung	TunnelRectangle
rechteckig breit	9 / 15 m	2-spurige Straßen in beide Richtungen mit Parkbuchten	TunnelRectangleWide

SCNX_LandscapeGeneration_700.cfg stellt die Vorlage für alle Tunnel. Dort wird festgeschrieben aus wie vielen und welchen Teilstücken der Tunnel besteht.



3.3 Brücke

Straßenbrücken oder –viadukte außerorts werden in drei Schritten definiert:

1. Ein spezieller CourseType definiert das Gelände der Brücke
2. Die umgebende Landschaft wird angepasst
3. Zusätzliche grafische Objekte (Brückenpfeiler) werden aufgestellt

CourseType

Die folgende CourseType-Definition, die im SCN-Abschnitt einzufügen ist, erzeugt eine Straßenbrücke mit zwei Fahrspuren:

```
CourseType Bruecke
{
    # distgerade, distanz, höhe, textur, transparenz
    Shoulder = {1.0, 0.01, -1.0, Concrete1, 0}; # Fahrbahnoberfläche und Seite

    ve = 70;
    LaneInfos =
    {
        (0, 3.5, 0.0, Towards, Tar, 1.0, 0.0, {(None, 0.0)}),
        (1, 3.5, 0.0, Onwards, Tar, 1.0, 0.0, {(None, 0.0)})
    };
    LaneTransitions =
    {
        (0, 0,
        {
            (BridgeRailing, -0.95, 1.0, Towards, 0,0), # Geländer
            (Line, 0.12, 0.15, Towards, 1.0, 0.0)
        }
        ),
        (0, 1,
        {
            (Line, 0.0, 0.15, Towards, 2.0, 6.0),
            (BridgeBottom, 0.0, 1.0, Towards, 0.0, 0.0) # Unterseite der Brücke
        }
        ),
        (1, 1,
        {
            (Line, -0.12, 0.15, Towards, 1.0, 0.0),
            (BridgeRailing, 0.95, 1.0, Onwards, 0,0) # Geländer
        }
        )
    };
};
```

Zu beachten ist, dass es sich beim Geländer um ein rein grafisches Objekt handelt. Wie auch bei Leitplanken hindert es den Fahrer nicht wirklich daran, die Straße zu verlassen, was insbesondere bei Simulatoren mit Bewegungssystem problematisch sein kann.

Abkommen von der Straße verhindern

Mit Hilfe des speziellen Hedgehogs **SCNX.OffRoad** kann verhindert werden, dass der Fahrer die Straße verlässt. Die Hedgehogs müssen im Course-Abschnitt eingefügt werden, hier beispielhaft für eine Brücke von 220 m Länge:

```
Hedgehogs = {
    # Verlassen der Strecke am Anfang der Brücke verbieten
    ( 3, DontCare, 0, SCNX, OffRoad, 0, OffRoadThreshold, 0.0),
    ( 3, DontCare, 1, SCNX, OffRoad, 0, OffRoadThreshold, 0.0),
    # am Schluss der Brücke wieder erlauben
```

```
(219, DontCare, 0, SCNX, OffRoad , 1, OffRoadThreshold, 0.0),
(219, DontCare, 1, SCNX, OffRoad , 1, OffRoadThreshold, 0.0)
};
```

Landschaftsanpassung

Durch die Landschaftsgeneratoren sollte sichergestellt sein, dass die Umgebung der Straße einige Meter unterhalb der Straße liegt. Beispielsweise kann hierzu im **Course**-Abschnitt eingefügt werden:

```
LandscapeTypeLeft <<beliebig>>
{
    Offset = -17;
    <<weitere Parameter wie gewünscht>>
};

LandscapeTypeRight <<beliebig>>
{
    Offset = -30;
    <<weitere Parameter wie gewünscht>>
};
```

Um zu verhindern, dass die Landschaftsoberfläche auf das Straßenniveau hochgezogen wird, muss noch der Parameter `LandscapeInfluence = 1` im Course-Abschnitt gesetzt werden.

Grafische Objekte

Bei längeren Viadukten können Brückenpfeiler als grafische Objekte unterhalb der Straße aufgestellt werden. Für eine 220 m lange Brücke kann beispielsweise im Course-Abschnitt folgendes eingefügt werden:

```
Objects =
{
    (environment.road.parts.bridgepillar, 30, 3.5),
    (environment.road.parts.bridgepillar, 30, -3.5),
    (environment.road.parts.bridgepillar, 80, 3.5),
    (environment.road.parts.bridgepillar, 80, -3.5),
    (environment.road.parts.bridgepillar,130, 3.5),
    (environment.road.parts.bridgepillar,130, -3.5),
    (environment.road.parts.bridgepillar,180, 3.5),
    (environment.road.parts.bridgepillar,180, -3.5)
};
```



Abbildung 6 Brücke

4 UMGEBENDER VERKEHR

4.1 Gegenverkehr auf der Landstraße

4.1.1 TrafficFlow (Quelle nicht ortsfest)

Im Folgenden wird ein dem SimCar entgegenkommender Trafficflow auf einer zweispurigen Straße definiert (vgl. dazu die Dokumentation Referenz TRFX, Kap. Fahrzeugquellen und –senken): Der Gegenverkehr befindet sich auf Spur 0 und wird 700m vor dem SimCar aufgesetzt (ist also nicht ortsfest) und 710m nach dem SimCar wieder „eingesammelt“ (die Senke muss weiter vom SimCar entfernt sein als die Quelle). Für dieses Beispiel muss in der **TRF-Definition** die Konfigurationsdatei `SILAB_TRFX_600.cfg` eingebunden sein. Der Gegenverkehr wird zu Beginn von Course1 aktiviert und bei Streckenmeter 2000 deaktiviert und zu Beginn von Course3 wieder aktiviert (Course1 und Course3 müssen im gleichen Modul definiert sein).

Die Angaben nach „FreeDriving“ legen jeweils Geschwindigkeiten und Beschleunigungswerte fest:

- Der Gegenverkehr startet mit 15 m/s (Truck) bzw. 20 m/s (Car, Van) (entspricht 54 km/h bzw. 72 km/h)
- Der Gegenverkehr fährt weiter mit 20 m/s (Truck) bzw. 25 m/s (Car, Van) (entspricht 72 km/h bzw. 90 km/h)
- Bei Änderungen der Zielgeschwindigkeit (Flowpoint „TargetSpeed“) wird mit 1m/s^2 beschleunigt bzw. mit 2m/s^2 verzögert, um diese Zielgeschwindigkeit zu erreichen.

```
TrafficFlow Gegenverkehr1
{
    Source Gegen1
    {
        Position = (SimCar, -700, 0);

        Vehicles =
        {
            (3, Car, (HazardAvoidance, 1), (FreeDriving, 20, 25, 1, 2)),
            (1, Van, (HazardAvoidance, 2), (FreeDriving, 20, 25, 1, 2)),
            (1, Truck, (HazardAvoidance, 2), (FreeDriving, 15, 20, 1, 2))
        };

        Parameters = (Gauss, 8, 3, 1236, 512);
    };
    Drain Gegen11
    {
        Position = (SimCar, 710, 0);
    };

    Flowpoints=
    {
        (Course1, 10, 1, SimCar, ActivateSource, (Gegen1)),
        (Course1, 1, 1, SimCar, ActivateDrain, (Gegen11)),
        (Course1, 2000, 1, SimCar, DeactivateSource, (Gegen1)),

        (Course3, 10, 1, SimCar, ActivateSource, (Gegen1)),
        (Course3, 1, 1, SimCar, ActivateDrain, (Gegen11)),
        (Course3, 1340, 1, SimCar, DeactivateSource, (Gegen1))
    };
};
```

Ein weiteres Beispiel für eine nicht ortsfeste Fahrzeugquelle ist die `SILABDemo_Country.cfg` (im Module `Country_Valley`) in der `SILABDemo`.

4.1.2 Einzelfahrzeuge

Im Folgenden werden drei dem SimCar entgegenkommende Einzelfahrzeuge auf einer zweispurigen Straße definiert (vgl. dazu die Dokumentation Referenz TRFX Kap. 2.2): Der Gegenverkehr befindet sich auf Spur 0 und wird bei 600 / 650 / 680 m aufgesetzt, wenn das SimCar den Streckenmeter 1 dieses Courses überfährt. Für dieses Beispiel muss in der **TRF-Definition** die Konfigurationsdatei `SILAB_TRFX_600.cfg` eingebunden sein. Der Gegenverkehr wird zu Beginn des Course „uf1“ aktiviert. Die Fahrzeuge starten mit 15m/s und beschleunigen mit 0.5m/s² auf 30m/s.

```
Vehicle gegen1
{
    Vehicle = CarVan;
    Position = (uf1, 600, 0);
    Behaviourmachine = {(HazardAvoidance, 3),(FreeDriving, 15, 30, 0.5, -0.5)};
    Flowpoints =
    {
        (uf1,1 , 1, SimCar, Activate)
    };
};
Vehicle gegen2
{
    Vehicle = CarVan;
    Position = (uf1, 650, 0);
    Behaviourmachine = {(HazardAvoidance, 3),(FreeDriving, 15, 30, 0.5, -0.5)};
    Flowpoints =
    {
        (uf1,1 , 1, SimCar, Activate)
    };
};
Vehicle gegen3
{
    Vehicle = Car;
    Position = (uf1, 680, 0);
    Behaviourmachine = {(HazardAvoidance, 3),(FreeDriving, 15, 30, 0.5, -0.5)};
    Flowpoints =
    {
        (uf1,1 , 1, SimCar, Activate)
    };
};
```

Dieses Beispiel findet man in der `SILABDemo_Country.cfg` (im Module `Country_Farmed`) in der `SILABDemo`.

4.2 Vorausfahrendes Fahrzeug

4.2.1 Vorausfahrendes abbiegendes Fahrzeug auf der Landstraße

Auf der Landstraße fährt ein Fahrzeug voraus, ändert zweimal seine Geschwindigkeit, biegt an einer Kreuzung rechts ab (dabei wird der Blinker automatisch gesetzt) und wird deaktiviert:

```
Vehicle Voraus1
{
    Vehicle = Van;
    Position = (CF1, 55, 1);
    Behaviourmachine =
    {
        (HazardAvoidance, 3), (FreeDriving, 22, 22, 0.5, -0.5)
    };
    Flowpoints =
```

```

{
    (CF0, 400, 1, SimCar, Activate),
    (CF1, 500, 1, Traffic, TargetSpeed, 27, 1.5, -0.5),
    (CF1, 1300, 1, Traffic, TargetSpeed, 17, 0.5, -0.5),
    (CF1, 1460, 1, Traffic, Right),
    (CF2, 100, 1, SimCar, Deactivate)
};
};

```

Für oben stehendes Beispiel muss in der RoadUserGroupTable die Fahrzeuggruppe „Bus-Quelle“ definiert sein.

4.2.2 Wanderbaustelle auf der Autobahn

Auf der rechten Spur einer 2-spurigen Autobahn befindet sich ein langsam fahrendes Fahrzeug (Pfeil blinkt nach links → IndicatorLeft = 1):



```

Vehicle WanderBS
{
    Vehicle = IvecoDaily_TrafficSafety_222_20;
    Position = (Wanderbaustelle, 950, 5);
    Behaviourmachine = {(HazardAvoidance, 1.5), (FreeDriving, 1, 5)};

    Flowpoints =
    {
        (Wanderbaustelle, 200, 5, SimCar, Activate),
        (Wanderbaustelle, 210, 5, SimCar, AutoIndicatorLightControl, 0),
        (Wanderbaustelle, 215, 5, SimCar, IndicatorLeft, 1),
        (Wanderbaustelle, 1900, 5, Traffic, TargetSpeed, 0)
    };
};

```

Dieses Beispiel für eine Wanderbaustelle findet man in der *SILABDemo_Autobahn.cfg* (im Module ABWanderbaustelle) in der SILABDemo. Für dieses Beispiel muss in der **TRF-Definition** zusätzlich die Datei *SILAB_TRFX_Construction_600.cfg* eingebunden werden.

4.3 Stau auf zweispuriger Autobahn

Folgendes Beispiel für einen kurzen Stau findet man in der *SILABDemo_Autobahn.cfg* (im Module Autobahn) in der SILABDemo: Dabei wird auf der rechten und der linken Spur jeweils ein Fahrzeug vor viele andere Fahrzeuge gesetzt, das sehr langsam wird und fast bis zu Stillstand abbremst, anschließend wieder beschleunigt → der Stau löst sich auf.

```
Vehicle reLKW
```

```

{
    Vehicle = Truck;
    Position = (Stau, 280, 5);
    Behaviourmachine = {(HazardAvoidance, 1),(FreeDriving, 15, 15)};
    Flowpoints =
    {
        (FreieFahrt_AB, 3200, 5, SimCar,Activate),
        (Stau, 200, 5, SimCar, TargetSpeed, 5),
        (Stau, 600, 5, SimCar, TargetSpeed, 0.4),
        (Stau, 630, 5, SimCar, TargetSpeed, 20),
        (Stau, 900, 5, SimCar, TargetSpeed, 25)
    };
};
Vehicle l1l1
{
    Vehicle = Car;
    Position = (Stau, 200, 4);
    Behaviourmachine = {(HazardAvoidance, 1),(FreeDriving, 15, 16)};
    Flowpoints =
    {
        (FreieFahrt_AB, 3200, 5, SimCar,Activate),
        (Stau, 200, 5, SimCar, TargetSpeed, 5),
        (Stau, 600, 5, SimCar, TargetSpeed, 0.5),
        (Stau, 615, 5, SimCar, TargetSpeed, 20),
        (Stau, 800, 5, SimCar, TargetSpeed, 35),
        (Stau, 1800, 5, SimCar, TargetSpeed, 45)
    };
};

```

Fahrzeuge hinter dem oben beschriebenen „Vehicle reLKW“ werden ebenso definiert, aber an anderer Position aufgesetzt (niedrigerer Streckenmeter, d.h. hinter „Vehicle reLKW“):

```

Vehicle re11
{
    Vehicle = Car;
    Position = (Stau, 200, 5);
    Behaviourmachine = {(HazardAvoidance, 1),(FreeDriving, 15, 15)};
    Flowpoints =
    {
        (FreieFahrt_AB, 3200, 5, SimCar,Activate),
        (Stau, 200, 5, SimCar, TargetSpeed, 5),
        (Stau, 600, 5, SimCar, TargetSpeed, 0.4),
        (Stau, 630, 5, SimCar, TargetSpeed, 20),
        (Stau, 910, 5, SimCar, TargetSpeed, 25)
    };
};

Vehicle re12
{
    Vehicle = Car;
    Position = (Stau, 150, 5);
    Behaviourmachine = {(HazardAvoidance, 1),(FreeDriving, 15, 15)};
    Flowpoints =
    {
        (FreieFahrt_AB, 3200, 5, SimCar,Activate),
        (Stau, 200, 5, SimCar, TargetSpeed, 5),
        (Stau, 600, 5, SimCar, TargetSpeed, 0.4),
        (Stau, 630, 5, SimCar, TargetSpeed, 20),
        (Stau, 920, 5, SimCar, TargetSpeed, 25)
    };
};

```

„Vehicle re13“ usw. werden analog definiert, auch auf Spur 4.

4.4 Stehende Fahrzeuge

4.4.1 Pannenfahrzeug auf der Autobahn

Auf dem Standstreifen (Spur 6) einer Autobahn, mit Warnblinkanlage (AutoIndicatorLightControl wird auf Null gesetzt, beide Blinker werden angeschaltet):

```
Vehicle Pannel
{
    Position = (FF_130_1, 460, 6);
    Vehicle = Car;
    Behaviourmachine =
    {
        (HazardAvoidance, 0.8),
        (FreeDriving, 0, 0, 0, 0)
    };
    Flowpoints =
    {
        (FF_130_1, 10, 0, SimCar, Activate),
        (FF_130_1, 11, 1, Traffic, AutoIndicatorLightControl, 0),
        (FF_130_1, 11, 0, SimCar, IndicatorLeft, 1),
        (FF_130_1, 12, 0, SimCar, IndicatorRight, 1)
    };
};
```

Dieses Beispiel für ein Pannenfahrzeug ist in der *SILABDemo_Autobahn.cfg* (im Module AB_Baustelle) in der SILABDemo.

4.4.2 Parkendes Fahrzeug

Parkendes Fahrzeug auf einer zweispurigen Straße (auf Spur 1 nach rechts versetzt): Um den Spurversatz nach rechts zu erreichen, darf die Zielgeschwindigkeit von *FreeDriving* nicht von Beginn an auf Null gesetzt werden.

```
RoadUser Hindernis
{
    Vehicle = Car;
    Position = (OD1, 200, 1);
    Behaviourmachine =
    {
        (FreeDriving, 0.2, 0, 1, -0.1),
        (CommandAction, 1, TRFC_DYLIMITR, 0.31, TRFC_DY, 0.5, TRFC_BRAKELIGHT, 0)
    };
    Flowpoints =
    {
        (OZ1, 1, 1, SimCar, Activate),
        (OZ1, 5, 1, SimCar, ActivateCommandAction, 1),
        (OZ1, 10, 1, SimCar, DistanceLess, 2000), # Bedingung für CommandAction
        (OZ1, 33, 1, SimCar, AutoBrakeLightControl, 0),
        (OZ1, 35, 1, SimCar, BrakeLight, 0), # Bremslicht wird ausgeschaltet
        (CF1, 500, 1, SimCar, Deactivate)
    };
};
```